

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Week 8 Laboratory Sheet

Soft Constraints and Optimization: From Over-Constrained CSPs to CP-SAT Objectives

Assoc. Prof. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Estimated time: 2 hours
Requirements: Terminal/shell access (macOS/Linux: Terminal, Windows: PowerShell), the course's shared Python 3.12 virtual environment from Week 1 (already carrying ortools), a text editor

Learning phases: 1 • Pre-instruction | 2 • Content | 3 • Participation | 4 • Assessment | 5 • Follow-through

1 • Pre-instruction — Why It Matters

Every solver you have run so far in this course either finds a solution or reports `INFEASIBLE` and stops. Today's five-talk conference schedule — the same T1–T3 instance Week 1 asked you to fit into two time slots — gains two more talks and quietly stops being satisfiable at all: no assignment to two slots keeps every shared-author pair apart. In the next two hours you build the machinery that turns "no valid schedule exists" into "here is the schedule that breaks the fewest, or the cheapest, rules," catch a genuinely silent bug in that machinery by running it yourself, and close by building one such model completely from scratch.

1 • Pre-instruction — Prior Knowledge

This sheet exercises all four units of this week's chapter: Max-CSP, weighted CSP and valued CSP (Unit 1), CP-SAT objective functions and reification (Unit 2), lexicographic optimization and Pareto fronts (Unit 3), and penalty variables for a complete soft-constraint model (Unit 4). It also leans on three earlier weeks: the formal CSP triple and Week 1's own five-talk instance (Week 1), the global-constraint vocabulary (Week 2), and the `model.Minimize()` objective-function API (Week 7). Keep the chapter open beside this sheet — each section below points to the unit it exercises.

Use this sheet as follows: in each step, complete the task described under `What will you do?`, then compare your own result against the `Expected result` box to confirm you are on track before moving on. Steps follow the chapter's four units in order, and a `Checkpoint` every few steps lets you confirm everything so far is working before you continue.

1 Setup: Confirming the Shared Environment

Step 1: Confirm this week needs no new package

What will you do? With the course's shared virtual environment `kod_ornekleri/.venv` active, you will print the installed `ortools` version to confirm the same package Week 1 installed is still there — this week is CP-SAT only, so no new package install is needed.

Step 1 (continued)

Why? This week reuses Week 1's `ortools` install unchanged (no Gurobi, no MiniZinc this week, per this week's scope); confirming the version before you start is what makes every `ciktikutusu` box below reproducible on your own machine, exactly as it was in Week 1 and Week 7.

Terminal

```
$ cd kod_ornekleri
$ source .venv/bin/activate
$ python -c "import ortools; print(ortools.__version__)"
```

Expected result: The command prints `9.14.6206`, the exact version Week 1 pinned for the whole course.

Expected Output

```
9.14.6206
```

Tip

If this command raises `ModuleNotFoundError`, your virtual environment is not active in this terminal — re-run `source .venv/bin/activate` from inside `kod_ornekleri` before continuing.

macOS / Linux

`source .venv/bin/activate` adds `(.venv)` to your shell prompt; every `python` command in the rest of this sheet then resolves to this environment's interpreter automatically.

Windows

In PowerShell, activate with `.venv\Scripts\Activate.ps1` instead.

2 Unit 1 — Over-Constrained Problems: Max-CSP, Weighted CSP, Valued CSP

(See Course Notes Week 8, Unit 1 — Over-Constrained Problems: Max-CSP, Weighted CSP, Valued CSP.)

Step 2: Split a fresh scenario into hard and soft, and choose Max-CSP or WCSP

What will you do? An exam office is scheduling four courses, C1–C4, into two exam slots (morning, afternoon). Courses share students, and a shared pair scheduled in the same slot is undesirable; the number of affected students for each pair is: C1–C2 (40 students), C1–C3 (15 students), C2–C3 (5 students), C3–C4 (2 students). On paper: (a) name the one constraint that must stay hard no matter what (it is not a candidate for violation at all), (b) explain why the four shared-student constraints cannot all be treated as hard, and (c) say whether a plain Max-CSP or a weighted CSP is the more informative model here, and why.

Why? This is Unit 1's WCSP definition applied to a fresh instance you have not seen worked out yet, in the same style Unit 1's own practice task applies it to the five-talk instance — writing out the hard/soft split and the Max-CSP-versus-WCSP choice yourself is what

Step 2 (continued)

turns following the chapter's argument into making this call on a new scenario.

Expected result: (a) The hard constraint is structural: each course is assigned to exactly one exam slot — this is built into the model's variables, not something a solution can "violate." (b) C1, C2 and C3 pairwise share students (C1–C2, C1–C3, C2–C3), forming a triangle; with only two slots, at least one of these three pairs is unavoidably scheduled together no matter how you assign them, so all four shared-student constraints must be relaxed to soft. (c) A weighted CSP is more informative: a plain Max-CSP would treat breaking the 40-student C1–C2 pair exactly as "bad" as breaking the 2-student C3–C4 pair, while a WCSP's weights let the objective prefer sacrificing the cheapest pair available. By hand, the cheapest edge inside the forced triangle is C2–C3 (weight 5, versus 40 and 15); one assignment achieving that minimum of 5 is C1 = morning, C2 = afternoon, C3 = afternoon, C4 = morning, which violates only C2–C3.

Step 3: Run overconstrained_timetable.py and match it to the WCSP definition

What will you do? You will run `overconstrained_timetable.py`, which extends Week 1's own three-talk instance with T4 and T5 (pairs T1–T3, T1–T4, T3–T4, T2–T5), and check its printed assignment and total violation count against the real captured output below.

Why? This is Unit 1's own worked instance, running as code for the first time: the same triangle structure (now on T1, T3, T4) you just reasoned about on paper in the exam-scheduling instance forces exactly one unavoidable violation here too, and seeing a real solver confirm that is what turns the paper argument into a verified fact.

The five slot variables and the reified violation Booleans, one per shared-author pair:

```
slots = {talk: model.NewIntVar(0, 1, f"slot_{talk}") for talk in TALKS}
return slots
```

```
violations = {}
for talk_a, talk_b in PAIRS:
   viol = model.NewBoolVar(f"viol_{talk_a}_{talk_b}")
   model.Add(slots[talk_a] == slots[talk_b]).OnlyEnforceIf(viol)
   model.Add(slots[talk_a] != slots[talk_b]).OnlyEnforceIf(viol.Not())
   violations[(talk_a, talk_b)] = viol
return violations
```

```
>_ Terminal

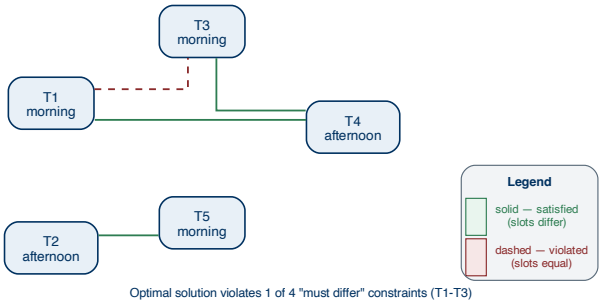
$ cd hafta08
$ python modeller/overconstrained_timetable.py
```

Expected result: A status of `OPTIMAL` and a total violation count of 1 — T1, T3 and T4 form the same kind of forced triangle you just identified on paper, so one clash among them is unavoidable no matter which two slots you choose.

Step 3 (continued)

Expected Output

```
Status: OPTIMAL
T1: morning
T2: afternoon
T3: morning
T4: afternoon
T5: morning
Total violations: 1
```



Optimal solution violates 1 of 4 "must differ" constraints (T1-T3)

Tip

If your terminal is still at the course root, `cd kod_ornekleri` first, then `cd hafta08` — every remaining command in this sheet is run from inside `kod_ornekleri/hafta08`.

Checkpoint — verify before moving on

You have split a fresh exam-scheduling scenario into hard and soft constraints entirely on paper, argued for WCSP over Max-CSP, and watched `overconstrained_timetable.py` confirm the same "one violation is unavoidable" result on Unit 1's own five-talk instance. If both match the `Expected result` boxes above, move on to Unit 2.

3 Unit 2 — Constraint Optimization Problems and Objective Functions in CP-SAT

(See Course Notes Week 8, Unit 2 — Constraint Optimization Problems and Objective Functions in CP-SAT.)

Step 4: Run the incomplete reified_extra_pair.py and find the undercount

What will you do? A starter file, `kod_ornekleri/hafta08/lab/reified_extra_pair.py`, adds a fifth shared-author pair, T2 and T4, to Unit 1's five-talk instance. Its reification for this new pair is deliberately incomplete: only the forward direction (`OnlyEnforceIf(viol_t2_t4)`) is wired up, and the reverse direction is left as a `TODO`. Run the file exactly as given, and read all three of its diagnostic lines carefully, not just the reported total.

Why? Unit 2's own practice task asks you to write both `OnlyEnforceIf` directions for a single reified pair; running a model that is missing only the second direction, on an instance

Step 4 (continued)

where that omission actually changes the outcome, is what makes "the solver could set `viol` to 0 even when the talks actually clash" concrete rather than a warning you take on faith.

The new pair's incomplete reification:

```python
viol_t2_t4 = model.NewBoolVar("viol_T2_T4")
model.Add(slots["T2"] == slots["T4"]).OnlyEnforceIf(viol_t2_t4)
# TODO: add the missing reverse direction, the same way
# add_original_violation_vars does for every other pair above:
# model.Add(slots["T2"] != slots["T4"]).OnlyEnforceIf(viol_t2_t4.Not())
return viol_t2_t4
```

Terminal

```
$ python lab/reified_extra_pair.py
```

Expected result: A status of `OPTIMAL` and a reported total of 1 — the same number Unit 1's plain four-pair model reported. But look at the last two diagnostic lines: T2 and T4 really do land in the same slot, and `viol_T2_T4` is still recorded as 0. The new pair is clashing for free, uncounted.

Expected Output

```
Status: OPTIMAL
T1: morning
T2: afternoon
T3: morning
T4: afternoon
T5: morning
Total violations (reported): 1
T2, T4 actually in the same slot: True
viol_T2_T4 recorded by the solver: 0
```

Common Pitfall

A reported total of 1 looks identical to Unit 1's original four-pair result, which makes it tempting to conclude that adding the T2–T4 pair "changed nothing." It did not change nothing — it changed the model into one that is silently wrong: with no cost attached to setting `viol_T2_T4 = 0` regardless of the true slot values, the solver is free to pick it that way even while T2 and T4 genuinely clash, because minimizing an under-constrained Boolean is trivially satisfied by leaving it at 0. A missing reverse direction does not raise an error; it produces a confidently printed, plausible-looking, wrong count.

Step 5: Complete the reverse direction on your own, and verify the fix

What will you do? In `lab/reified_extra_pair.py`, where the `TODO` comment says to, add the missing reverse-direction call: `model.Add(slots["T2"] != slots["T4"]).OnlyEnforceIf(viol_t2_t4.Not())`. Then re-run the script.

Why? This is Unit 2's own "Now You Try" pattern — writing the two `OnlyEnforceIf` calls that correctly link a reified Boolean to a condition — but here you complete it on a running model and watch a silent undercount disappear, rather than only checking your two lines against a written answer.

Step 5 (continued)

Terminal

```
$ python lab/reified_extra_pair.py
```

Expected result: The reported total is still 1, but now for the right reason: T2 and T4 no longer share a slot, so `viol_T2_T4` correctly reads 0, and the sole violation is again T1–T3, exactly as in Unit 1's plain four-pair model. Fixing the reification did not need to make the number worse — it only needed to make the number *true*.

Expected Output

```
Status: OPTIMAL
T1: morning
T2: morning
T3: morning
T4: afternoon
T5: afternoon
Total violations (reported): 1
T2, T4 actually in the same slot: False
viol_T2_T4 recorded by the solver: 0
```

Tip

This particular instance has enough slack elsewhere (T2 and T5 are not tied to the forced T1–T3–T4 triangle) that the solver can always route around a single extra reified pair at zero extra cost — that is a property of this instance's structure, not a general rule; a differently connected instance could easily force the reported total to increase once the second direction is added correctly.

Checkpoint — verify before moving on

You have run a model whose missing reverse-direction `OnlyEnforceIf` call produced a plausible-looking but wrong violation count, then fixed it yourself and watched the diagnostic lines confirm the repair. You now have direct, hands-on evidence for Unit 2's warning that reification needs both directions, not just one.

4 Unit 3 — Lexicographic and Multi-Objective Optimization, Pareto Fronts

(*See Course Notes Week 8, Unit 3 — Lexicographic and Multi-Objective Optimization, Pareto Fronts.*)

Step 6: Run lexicographic_timetable.py and confirm stage 1's optimum is protected

What will you do? You will run `lexicographic_timetable.py`, which first minimizes total violations on Unit 1's five-talk instance (stage 1), then pins that optimal value as a hard constraint before minimizing a second objective, the number of talks scheduled in the afternoon (stage 2). Check both stages' printed values, and confirm the stage-1 violation count is not made worse by stage 2.

Step 6 (continued)

Why? This is the exact two-step recipe Unit 3's own practice task asks you to write by hand — solve, pin, solve again — now running as real code; verifying that stage 2 never trades away stage 1's proven optimum is the one property that makes an optimization genuinely *lexicographic* rather than two independent, possibly conflicting solves.

Pinning stage 1's optimum as a hard constraint before stage 2 begins:

```
model.Add(total_violations == v1_star)
afternoon_count = sum(slots[talk] for talk in TALKS)
```

Terminal

```
$ python modeller/lexicographic_timetable.py
```

Expected result: Stage 1 reports an optimal total violation count of 1 (matching Unit 1's own run); stage 2, solved on a model with that value pinned, reports an optimal afternoon count of 2.

Expected Output

```
Stage 1 optimal total violations: 1
Stage 2 optimal afternoon count (violations pinned at 1): 2
T1: morning
T2: morning
T3: morning
T4: afternoon
T5: afternoon
```

Tip

Compare stage 2's assignment against Unit 1's own single-objective run: both report a total violation count of 1, but stage 2's assignment is free to differ in *which* talks land in the afternoon, since only the violation count — never a specific assignment — was pinned.

Step 7: Run pareto_front_demo.py and identify the Pareto front from its own output

What will you do? You will run `pareto_front_demo.py`, which enumerates every one of the 32 feasible slot assignments for the five-talk instance, records each one's (violations, afternoon count) pair, and filters the Pareto-optimal subset. From the printed candidate list, manually verify that the pair `(2, 2)` is *not* on the front, by naming one printed candidate that dominates it.

Why? Unit 3 defines Pareto dominance abstractly; checking one specific non-front candidate against the tool's own full enumeration, rather than only reading the printed `<- Pareto front` markers, is what confirms you can apply the dominance test yourself, not just trust the script's filter.

The dominance test itself, applied to every pair of candidates:

```
at_least_as_good = candidate_a[0] <= candidate_b[0] and candidate_a[1] <= candidate_b
    [1]
strictly_better = candidate_a[0] < candidate_b[0] or candidate_a[1] < candidate_b[1]
return at_least_as_good and strictly_better
```

Step 7 (continued)

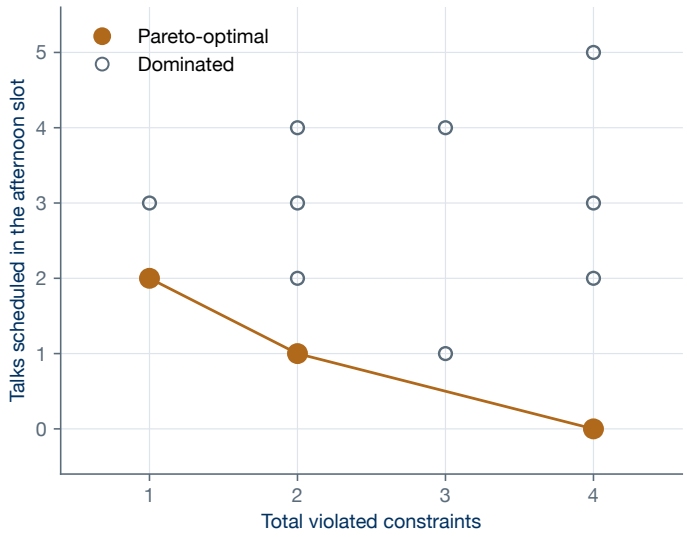
Terminal

```
$ python modeller/pareto_front_demo.py
```

Expected result: Every enumerated `(violations, afternoon_count)` pair, with the Pareto-optimal ones marked; the front itself is exactly the three pairs `(1, 2)`, `(2, 1)` and `(4, 0)`.

Expected Output

```
All candidates (violations, afternoon_count):
  (1, 2) <- Pareto front
  (1, 3)
  (2, 1) <- Pareto front
  (2, 2)
  (2, 3)
  (2, 4)
  (3, 1)
  (3, 4)
  (4, 0) <- Pareto front
  (4, 2)
  (4, 3)
  (4, 5)
Pareto front:
  (1, 2)
  (2, 1)
  (4, 0)
```



Expected result: For the manual check: candidate `(2, 2)` is dominated by `(1, 2)` — both share the same afternoon count (2), but `(1, 2)` is strictly better on violations (1 versus 2), so `(1, 2)` satisfies Unit 3's dominance definition against `(2, 2)` and `(2, 2)` correctly stays off the front.

Checkpoint — verify before moving on

`lexicographic_timetable.py` confirmed stage 2 never sacrifices stage 1's proven violation count of 1, and you traced the dominance test by hand on `pareto_front_demo.py`'s own full candidate list, confirming why `(2, 2)` does not belong on the three-point Pareto front. You are ready for the week's most complete model.

5 Unit 4 — Penalty Variables and Soft Global Constraints

(See Course Notes Week 8, Unit 4 — Penalty Variables and Soft Global Constraints.)

Step 8: Run nurse_preferences.py and verify the hard/soft split

What will you do? You will run `nurse_preferences.py` — a 3-nurse, 2-day, 2-shift roster with hard coverage and one-shift-per-day rules, plus soft evening-shift preferences weighted N1 = 3, N2 = 2, N3 = 1 — and check the printed schedule against every hard rule by hand: exactly one nurse per shift-slot, and no nurse working two shifts on the same day.

Why? This is Unit 4's complete worked model, combining Unit 1's weighted-violation framing with Unit 2's reification pattern for the first time inside one program with real hard constraints alongside the soft ones; checking the hard rules by hand on the printed output is the same kind of independent verification this week's `pytest` gate performs automatically.

The hard coverage and one-shift-per-day constraints, never reified:

```
for day in DAYS:
    for shift_type in SHIFT_TYPES:
        model.Add(sum(shift[nurse, day, shift_type] for nurse in NURSES) == 1)
for nurse in NURSES:
    for day in DAYS:
        model.Add(sum(shift[nurse, day, shift_type] for shift_type in SHIFT_TYPES) <=
            1)
```

The reified preference Booleans, one per nurse per day:

```
pref_violated = {}
for nurse in NURSES:
    for day in DAYS:
        violated = model.NewBoolVar(f"pref_violated_{nurse}_{day}")
        model.Add(shift[nurse, day, "evening"] == 1).OnlyEnforceIf(violated)
        model.Add(shift[nurse, day, "evening"] == 0).OnlyEnforceIf(violated.Not())
        pref_violated[(nurse, day)] = violated
return pref_violated
```

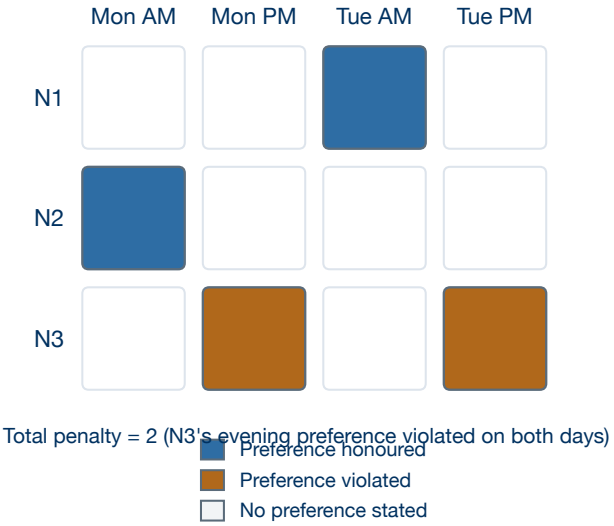
```
Terminal
$ python modeller/nurse_preferences.py
```

Expected result: A status of `OPTIMAL`; both evening slots are covered by N3, the cheapest weight to violate, and the total penalty is 2 (two evening shifts at weight 1 each). Every shift-slot has exactly one nurse, and no nurse works both shifts on the same day.

Step 8 (continued)

Expected Output

```
Status: OPTIMAL
N1: Mon:off  Tue:morning
N2: Mon:morning  Tue:off
N3: Mon:evening  Tue:evening
Total penalty: 2
```



Step 9: Edit one preference weight in nurse_preferences_lab.py and observe the shift

What will you do? A copy of the nurse model, `kod_ornekleri/hafta08/lab/nurse_preferences_lab.py`, reuses every hard constraint and every reification call from `nurse_preferences.py` unchanged, exposing only one editable line: `NURSE_LAB_WEIGHTS`. Change N3's weight from 1 to 5 — making N3 more expensive to inconvenience than N2 (weight 2) — and re-run.

Why? Both evening shifts landed on N3 in the previous step only because N3 was the *cheapest* nurse to inconvenience; changing that one number, without touching any constraint or any reification call, is what demonstrates that the weighted objective — not the hard constraints — is what decides *who* absorbs an unavoidable preference violation.

The one line you are meant to edit:

```
# Original weights (modeller/nurse_preferences.py): {"N1": 3, "N2": 2, "N3": 1}.
# Edit N3's weight below, then re-run, to see the schedule change.
NURSE_LAB_WEIGHTS = {"N1": 3, "N2": 2, "N3": 1}
```

Terminal

```
$ python lab/nurse_preferences_lab.py
```

Expected result: With N3 now the *most* expensive nurse to inconvenience (weight 5, versus N2's 2), both evening shifts move to N2 instead, and the total penalty rises from 2 to 4 (two evening shifts at weight 2 each) — still cheaper than routing them through N3 at weight 5, or splitting them between N2 and N3 at weight $2 + 5 = 7$.

Step 9 (continued)

Expected Output

```
Status: OPTIMAL
N1: Mon:off  Tue:morning
N2: Mon:evening  Tue:evening
N3: Mon:morning  Tue:off
Total penalty: 4
```

Common Pitfall

It is tempting to assume that raising one nurse's weight must raise the *whole* schedule's penalty roughly in proportion to that one number, but the effect is entirely structural: the objective still routes every unavoidable violation through whichever nurse is currently cheapest, and the total penalty is that cheapest nurse's weight multiplied by however many violations remain unavoidable — here, still exactly two evening shifts, just reassigned to a different nurse.

Checkpoint — verify before moving on

`nurse_preferences.py` confirmed every hard rule holds by hand, and editing a single weight in `nurse_preferences_lab.py` moved both unavoidable evening violations from N3 to N2 and raised the total penalty from 2 to 4 — with no constraint or reification call touched. You have now seen every piece of this week's chapter run as real, verified CP-SAT code.

6 Capstone — Build a Complete Soft-Constraint Model From Scratch

Step 10: Build the shared-equipment WCSP model entirely from scratch

What will you do? A shared lab has four booking requests, R1–R4, for two time windows (morning, afternoon). One hard rule applies: R4's instrument is only calibrated in the afternoon, so R4 must be scheduled in the afternoon no matter what. Three further requests share equipment and would rather not overlap, each with a weight for how costly an overlap is: R1–R2 (weight 3), R1–R3 (weight 1), R2–R3 (weight 2), R2–R4 (weight 2). Write, in a new file (no starter file this time), a complete CP-SAT model: one integer slot variable per request, the one hard constraint fixing R4's slot, one reified violation Boolean per weighted pair (both `OnlyEnforceIf` directions), and a weighted `model.Minimize()` objective. Solve it and verify your printed result against the value you compute by hand below *before* you run it.

Why? This is this week's integrative step: translating a written scenario — with one genuinely hard constraint sitting alongside several weighted soft ones, exactly Unit 4's recipe — into a verified, running CP-SAT program without a paper model or a starter file handed to you first, the same pattern Week 1's and Week 7's capstones used for their own tools, and the first model this week that combines a real hard constraint with a weighted soft objective outside the nurse example.

Expected result: By hand: R4 is fixed to the afternoon. Setting R2 to the morning slot avoids both R2–R4 (weight 2) and, if R1 is set to the afternoon, R1–R2 (weight 3) as well. That leaves R1 and R3 both in the afternoon, violating only R1–R3 (weight 1) — and R2–R3 is avoided too, since R2 is morning and R3 is afternoon. No cheaper combination exists: the two most expensive pairs, R1–R2 (3) and R2–R4 (2), can both be avoided at once by

Step 10 (continued)

placing R2 alone in the morning, leaving only the single lightest pair, R1–R3 (1), unavoidably violated. The optimum is a total penalty of **1**, with R1 = afternoon, R2 = morning, R3 = afternoon, R4 = afternoon.

Expected Output

```
Status: OPTIMAL
R1: afternoon
R2: morning
R3: afternoon
R4: afternoon
Total penalty: 1
```

Tip

Build the model with the same four-piece recipe every script this week has used: one `NewIntVar(0, 1, ...)` per request, one `model.Add(slot["R4"] == 1)` for the hard rule, one `NewBoolVar` plus both `OnlyEnforceIf` directions per weighted pair (reusing `nurse_preference_vars`'s pattern), then `model.Minimize(sum(weight * viol for ...))` and `CpSolver().Solve(model)`.

Common Pitfall

Forgetting the reverse `OnlyEnforceIf` direction on even one weighted pair reproduces exactly the silent undercount you diagnosed in Unit 2 — a plausible-looking total penalty that does not actually match the printed slot assignment. After you get `OPTIMAL`, check every weighted pair's actual slot values against its violation Boolean by hand, the same way `reified_extra_pair.py`'s diagnostic lines did for you earlier.

☑ Checkpoint — verify before moving on

Your from-scratch shared-equipment model reports a total penalty of 1, with R1, R3 and R4 in the afternoon and R2 alone in the morning — matching the number and assignment you derived on paper before running it. This is the same variables-hard-constraints-weighted-objective-solve pattern you have now built or repaired five times today, the last time entirely from a written description rather than a paper model or a starter file someone else prepared for you.

Troubleshooting

Symptom	Likely cause	Fix
`ModuleNotFoundError: No module named 'ortools'`	The virtual environment is not active in this terminal	Re-run `source .venv/bin/activate` (or `Activate.ps1` on Windows) from inside `kod_ornekleri`
Running any `modeller/*.py` script prints `No such file or directory`	The command was run from the wrong working directory	Run `pwd` and confirm you are inside `kod_ornekleri/hafta08` before running any script
Your fixed `reified_extra_pair.py` still reports `viol_T2_T4` as 0 while T2 and T4 share a slot	The reverse `OnlyEnforceIf` call was added to the wrong Boolean, or added without the `.Not()`	Re-check the line against `extra_pair_setup`'s loop, which applies the identical pattern to every original pair
Both evenings still route to N3 after you raised its weight	The edited dictionary was saved under a different variable name than `NURSE_LAB_WEIGHTS`, so `solve()` is still reading the unedited default	Confirm the edited dictionary keeps the exact name `NURSE_LAB_WEIGHTS` used inside `solve()`
Your from-scratch capstone model reports `INFEASIBLE`	The hard constraint on R4 was written with the wrong slot index, conflicting with another constraint that also pins a value	Print `model.Validate()` or re-read the hard constraint against the scenario text: R4 must equal 1 (afternoon), nothing else is fixed
`pytest -q` reports failures unrelated to your own edits	`modeller/` files were edited directly instead of the `lab/` starter files	Restore `modeller/*.py` from the course repository; only files inside `lab/` are meant to be edited this week

5 · Follow-through **Transfer — Where Else?**

The habit you practiced five times today — state which rules must stay hard, reify every soft one into a Boolean the objective can see, weight it, and only then call `model.Minimize()` — is not disposable lab work: Week 12 reuses this exact nurse-rostering model, now with interval variables standing in for shifts, and Week 15 revisits the over-constrained timetabling instance you ran today to ask a different question about a hard-constraint-only version of the same schedule. Beyond this course, whatever real scheduling, rostering or resource-allocation problem you meet next — in a paper, in your thesis, or in a future job — rarely has every rule negotiable and rarely has every rule sacred; the first useful question is almost always the one Unit 1 asked first: which rules are truly hard, which are soft, and what does each violation actually cost.